ScreenAI Project Report

A detailed overview of the AI-assisted recruitment screening platform, including architecture, roles, workflows, security, dashboards, assessment system, current status, and recommended next steps.

Project: ScreenAI

Workspace: D:\internship\ScreenAI

Generated: 25 June 2026

ScreenAI helps HR teams publish jobs, collect applications, analyze resumes, manage interviews, track hiring progress, and run coding assessments for candidates.

1. Executive Summary

ScreenAI is a recruitment management platform with AI support. It combines public application collection, recruiter workflows, admin governance, AI resume analysis, and browser-based coding assessments into one system.

- Recruiters can create jobs, review candidates, schedule interviews, send assessments, and manage hiring decisions.
- Admins can manage recruiters, monitor global platform data, inspect audit logs, and enforce security controls.
- Candidates can apply publicly and complete coding assessments through secure links.
- The newest major feature is the take-home coding assessment workflow, which is still the area needing the most finishing polish.

2. Technology Stack

Layer	Technology
Frontend	React, Vite, React Router, Axios, Bootstrap, Material UI, MUI DataGrid, Monaco Editor
Backend	Django, Django REST Framework, Simple JWT, django-cors-headers
Database	SQLite for local development; production should use PostgreSQL or another managed database
AI / Parsing	Gemini API via google.generativeai, pdfplumber
Assessment Runtime	Docker sandbox for safe code execution and grading
Email	Brevo API for transactional assessment invitation emails

3. Project Structure

The project is separated into a Django backend and a React frontend. The backend exposes APIs and performs secure business logic. The frontend provides dashboards and candidate-facing pages.

```
ScreenAI/
■■■ backend/ - Django API and business logic
```

```
■ ■■■ accounts/ - authentication, roles, audit logs, security state
■ ■■■ jobs/ - job posting and public job links
■ ■■■ applications/ - applications, resumes, interviews, progression
■ ■■■ assessments/ - take-home assessment templates, invitations, answers, grading
■ ■■■ ai_engine/ - Gemini resume parsing and scoring
■ ■■■ screenai/ - Django project settings and routing
■■■ frontend/ - React dashboard and public/candidate UI
```

4. User Roles

Role	Main Responsibilities
Admin	Create recruiters, suspend/reactivate users, reset credentials, view global directories, inspect activity logs, and gover
HR Recruiter	Create jobs, review candidates, manage interviews, send assessments, evaluate submissions, and update hiring out
Candidate	Apply through public links, upload resume, receive assessment link, write code in browser, and submit final answers.

5. Backend Modules

5.1 accounts

- Handles user profiles, roles, JWT authentication, security state, forced password changes, and audit logging.
- Includes token revocation support when a recruiter is suspended or credentials are reset.
- Audit logs are append-only and sanitize sensitive metadata such as passwords and tokens.

5.2 jobs

- Stores job postings, public application tokens, deadlines, status, application-form availability, and job metadata.
- Recruiters can create, edit, close, reopen, and share jobs.
- Job mutations are audited for admin traceability.

5.3 applications

- Stores candidate applications, resumes, AI scoring data, interviews, and post-hire progression.
- CandidateIdentity groups public/registered/anonymous candidate identities separately from individual applications.
- Supports candidate filtering, interview pipeline, resume streaming, and hiring decision updates.

5.4 ai_engine

- Uses Gemini to analyze resumes, extract candidate details, and generate AI scores and recommendations.
- The current google.generativeai package emits a deprecation warning and should later be migrated to google.genai.

5.5 assessments

- Manages assessment templates, questions, candidate assignments, email delivery records, candidate answers, submissions, and grading results.
- Supports secure invitation tokens, browser-based coding workspace, Docker-backed run/evaluate operations, and recruiter result review.

- This module is active and still needs refinement around fully data-driven question templates and LeetCode-style sample test output.

6. Frontend Dashboards

6.1 HR Dashboard

- Main recruiter workspace with Overview, Candidates, Jobs, Assessments, and Profile sections.
- Overview metric cards use interactive popups and open unified dialogs for detailed items.
- Candidate workspace dialog is now the main unified candidate-detail surface.
- Jobs directory uses a Material UI DataGrid with filters and row actions.

6.2 Admin Dashboard

- Provides platform metrics, recruiter management, global job/application/interview/hire directories, and audit/activity views.
- Includes recruiter drawer for profile, jobs, applications, hires, interviews, and activity.
- Supports recruiter suspension, activation, and credential reset workflows.

6.3 Candidate Assessment Page

- Candidate-facing browser coding workspace with Monaco Editor.
- Shows questions, problem statement, examples, timer, Run Code, Save Progress, and Final Submit.
- Needs final polish so all visible test cases/examples come from recruiter-created template data instead of hardcoded/seed assumptions.

7. Core Workflows

Workflow	Flow
Public Application	Recruiter creates job -> copies public link -> candidate applies -> resume is parsed -> AI score appears.
Recruiter Review	Recruiter opens Candidates -> candidate workspace -> reviews AI/resume -> shortlist/reject/hire.
Interview	Recruiter schedules interview -> tracks rounds -> records outcome.
Post-Hire	Candidate is hired -> progression stages are recorded and tracked.
Assessment	Recruiter sends assessment -> candidate writes code in browser -> submits -> recruiter evaluates -> Docker run

8. Security and Governance

- JWT authentication with role-aware backend permissions.
- Immediate session revocation when a recruiter is suspended or credentials are reset.
- Forced password change flow after admin credential reset.
- Protected resume streaming, restricted to staff/admin or the owning recruiter.
- Audit logs for admin actions, job changes, application changes, interview updates, progression updates, resume access, and assessment events.
- Private assessment tokens are stored as digests, not raw tokens.

- Docker sandboxing prevents candidate code from running directly on the host system.

9. Assessment System Status

The assessment system is functional enough for local experimentation: recruiters can assign assessments, candidates can open the browser workspace, write code, run code, submit, and view graded results. However, this area is still being actively refined.

Area	Current State
Template creation	Works, but Add Question does not yet collect all candidate-page fields such as examples and structured vis
Candidate coding page	Exists with editor, timer, question list, run button, and submit button.
Run Code	Needs LeetCode-style sample test output: input, actual return value, expected output, pass/fail, and separa
Evaluation	Uses Docker hidden tests. Docker Desktop must be running locally.
Result display	Needs correction so zero-score or failed tests do not display as Passed.
Email	Brevo integration is planned/partially wired and needs live environment configuration for real delivery.

10. Known Issues and Risks

- Assessment template fields are incomplete for fully dynamic question generation.
- Some assessment examples/sample tests may still be seed-generated or hardcoded.
- Run Code should show function return values without requiring print statements.
- Result status logic must distinguish between 'graded' and actually 'passed'.
- Docker must be running for local Run Code and evaluation.
- Background-thread evaluation is acceptable for development but should become a proper worker system for production.
- SQLite is acceptable locally, but production should use a stronger database such as PostgreSQL.

11. Deployment Readiness

Already Improved

- Hardcoded localhost URLs moved to environment variables.
- Admin security and session revocation improved.
- Audit logs added.
- Resume access protected.
- Dashboards refactored and visually unified.
- Candidate workspace dialog unified.
- Browser-based assessment flow introduced.

Before Production

- Finalize assessment template manager and sample test data structure.
- Fix Run Code sample output and grading pass/fail display.

- Configure Brevo API key, sender, webhook secret, and production frontend URL.
- Move from SQLite to a production database.
- Configure HTTPS, allowed hosts, CORS, private media storage, and proxy settings.
- Remove dev-only buttons/labels from recruiter assessment UI.
- Run full regression tests and frontend build.

12. Recommended Next Steps

Priority	Action
1	Fix assessment template manager so questions include function name, visible sample tests, examples, hidden tests, expecte
2	Fix Run Code to show actual function output and visible sample test results, with console output separate.
3	Fix assessment result display so failed tests do not show as Passed.
4	Test with a fresh candidate from application to assessment result.
5	Configure Brevo live email delivery.
6	Commit the complete assessment feature once stable.

13. Conclusion

ScreenAI has grown into a strong recruitment platform with admin governance, recruiter workflows, AI resume screening, secure application handling, and a developing coding assessment system. The strongest current product value is the unified recruiter/admin workflow. The main unfinished area is assessment template completeness and LeetCode-style candidate test execution feedback.